

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Теоретические основы баз данных
Отчет о выполнении работы на тему
«Нереляционные возможности PostgreSQL»

Студент,
группы 5130201/40102

Адиатуллин Т. Р. _____

Руководитель,
доцент к.т.н., доцент

Попов С. Г. _____

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Реферат

Содержание

Введение	4
1 Постановка задачи	5
2 Представления	6
2.1 Запрос для создание представления	6
2.2 Выполнение запросов к представлению	7

Введение

1. Постановка задачи

2. Представления

В данном разделе создаётся представление по ранее созданной базе данных предметной области «Геомониторинг автопарка каршеринга» и выполняются запросы к нему.

Постановка задачи

- Создать представление на основе таблиц базы данных по предметной области Геомониторинг автопарка каршеринга.
- Написать и выполнить простой запрос к созданному представлению.
- Написать и выполнить сложный запрос к созданному представлению.

2.1. Запрос для создание представления

Для каждого автомобиля подсчитать количество тревожных событий и число парковочных зон, в которых этот автомобиль парковался.

```
1 CREATE VIEW car_stats AS
2 SELECT
3     c.car_id,
4     c.reg_number,
5     COUNT(DISTINCT a.alert_event_id) AS alert_count,
6     COUNT(DISTINCT pz.parking_zone_id) AS parking_count
7 FROM car c
8 LEFT JOIN alert_event a ON c.car_id = a.car_id
9 LEFT JOIN parking_session ps ON c.car_id = ps.car_id
10 LEFT JOIN parking_zone pz ON ps.parking_zone_id =
    pz.parking_zone_id
11 GROUP BY c.car_id, c.reg_number;
```

Listing 1: Создание представления car_stats

car_id	reg_number	alert_count	parking_cou..
1	A001AA77	3	15
2	A002AA78	3	19
3	A003AA79	2	12
4	A004AA80	3	19
5	A005AA81	1	10
6	A006AA77	1	19
7	A007AA78	1	10
8	A008AA79	3	10
9	A009AA80	2	15
10	A010AA81	3	11
11	A011AA77	2	19
12	A012AA78	3	13
13	A013AA79	1	13
14	A014AA80	1	10
15	A015AA81	2	14

Рис. 1: Содержимое представления car_stats

В качестве основной таблицы выбрана таблица `car`. `LEFT JOIN` позволяют включить в представление каждый автомобиль, в том числе автомобиль без тревожных событий и парковочных сессий. Использование `COUNT(DISTINCT ...)` необходимо, поскольку соединение событий и парковочных сессий может образовать повторяющиеся строки.

Реляционная формула представления:

$$\gamma_{\begin{array}{l} \text{car_id, reg_number;} \\ \text{COUNT(alert_event_id) \rightarrow alert_count;} \\ \text{COUNT(parking_zone_id) \rightarrow parking_count} \end{array}} \left(\begin{array}{l} \text{car} \\ \bowtie (left)_{\text{car_id}} \text{ alert_event} \\ \bowtie (left)_{\text{car_id}} \text{ parking_session} \\ \bowtie (left)_{\text{parking_zone_id}} \text{ parking_zone} \end{array} \right)$$

2.2. Выполнение запросов к представлению

Простой запрос

Вывести автомобили, которые парковались более чем в N разных зонах и имеют хотя бы два тревожных события.

```

1 SELECT
2   c.reg_number,
3   c.alert_count,
4   c.parking_count
5 FROM
6   car_stats c
7 WHERE
8   c.parking_count > 17
9   AND
10  c.alert_count > 1;
```

Listing 2: Простой запрос

Результат выполнения простого запроса представлен на рисунке 2.

	reg_number	alert_count	parking_count
1	A002AA78	3	19
2	A004AA80	3	19
3	A011AA77	2	19
4	A016AA77	3	19
5	A024AA80	3	19
6	A027AA78	2	20
7	A041AA77	2	18
8	A053AA79	3	19
9	A067AA78	3	18
10	A072AA78	3	19
11	A073AA79	3	20
12	A088AA79	3	19
13	A089AA80	3	19
14	A090AA81	3	20
15	A091AA77	2	20
16	A093AA79	2	18
17	A097AA78	3	18
18	A099AA80	2	18
19	A110AA81	3	20
20	A121AA77	2	18
21	A124AA80	2	18

Рис. 2: Результат выполнения простого запроса к представлению

Время выполнения запроса:

Execution time: 0,207ms

Planning time: 0,291ms

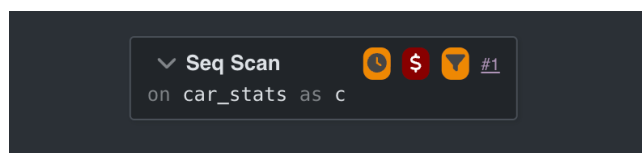


Рис. 3: Последовательность выполнения простого запроса

Таблица **car_stats** читается последовательно. Во время чтения проверяются условия на количество парковочных зон и тревожных событий, после чего в результат попадают только подходящие автомобили.

Реляционная формула запроса:

$$\pi_{\text{reg_number}, \text{alert_count}, \text{parking_count}} \left(\sigma_{\text{parking_count} > 17 \wedge \text{alert_count} > 1} \left(\text{car_stats} \right) \right)$$

Сложный запрос

Для выполнения сложного запроса к созданному представлению был сформулирован запрос: вывести автомобили, которые парковались менее чем в N разных зонах и имеют необработанные тревожные события заданного типа.

```
1 SELECT
2   DISTINCT
3   c.reg_number,
4   c.alert_count
5 FROM
6   car_stats c
7 JOIN
8   alert_event a ON c.car_id = a.car_id
9 JOIN
10  alert_event_process_status s ON a.status_id = s.status_id
11 JOIN
12  alert_event_type t ON a.alert_event_type_id =
13    t.alert_event_type_id
14 WHERE
15   c.parking_count < 18
16   AND
17   s.name = 'Новое'
18   AND
19   t.name = 'Потеря связи';
```

Listing 3: Сложный запрос

Результат выполнения сложного запроса представлен на рисунке 4. В результате выполнения было выведено 19 автомобилей.

	reg_number	alert_count
1	A010AA81	3
2	A021AA77	1
3	A034AA80	2
4	A040AA81	2
5	A044AA80	1
6	A045AA81	2
7	A058AA79	3
8	A063AA79	3
9	A132AA78	2
10	A138AA79	2
11	A141AA77	3
12	A157AA78	3
13	A173AA79	3
14	A189AA80	3
15	A197AA78	3

Рис. 4: Результат выполнения сложного запроса к представлению

Время выполнения запроса:
 Execution time: 0,564ms
 Planning time: 1,5ms

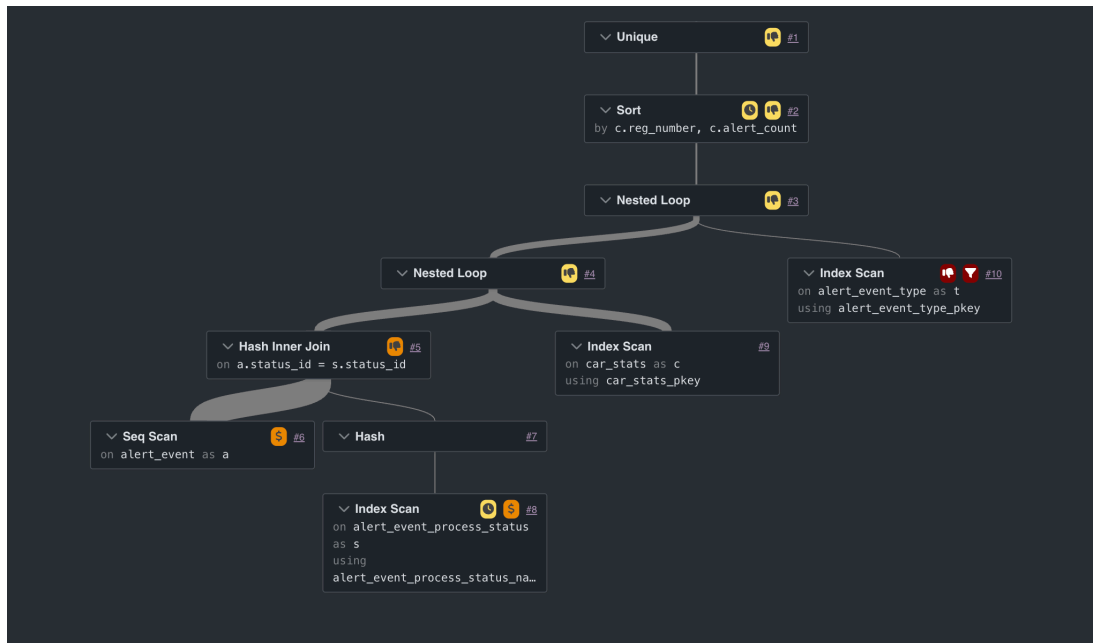


Рис. 5: Последовательность выполнения сложного запроса

Сначала последовательно читается таблица **alert_event**, а по индексу находятся статусы обработки с именем «Новое». Эти таблицы соединяются через Hash Join. Затем для каждого подходящего события по первичному ключу находятся строка в **car_stats** и его тип события. В конце результат сортируется, а оператор **Unique** удаляет повторяющиеся автомобили.

Реляционная формула запроса:

$$\begin{aligned}
 &\pi_{\text{reg_number}, \text{alert_count}} \left(\right. \\
 &\quad \sigma_{\text{parking_count} < 18 \wedge \text{s.name} = \text{«Новое»} \wedge \text{t.name} = \text{«Потеря связи»}} \left(\right. \\
 &\quad \quad \left(\right. \\
 &\quad \quad \quad \text{car_stats} \\
 &\quad \quad \quad \bowtie_{\text{car_id}} \text{alert_event} \\
 &\quad \quad \quad \bowtie_{\text{status_id}} \text{alert_event_process_status} \\
 &\quad \quad \quad \bowtie_{\text{alert_event_type_id}} \text{alert_event_type} \\
 &\quad \quad \left. \right) \\
 &\quad \left. \right) \\
 &\left. \right)
 \end{aligned}$$

3. Триггеры

Постановка задачи

На основе представления **car_stats** создаётся таблица с теми же показателями. Триггеры должны поддерживать согласованность её данных при изменении таблиц **car** и **parking_session**.

3.1. Создание таблицы статистики

Сначала представление переименовывается в **car_stats_view**, после чего создаётся таблица **car_stats** и заполняется данными из представления. Порядок операций важен: представление и таблица не могут иметь одинаковое имя.

```
1 ALTER VIEW car_stats RENAME TO car_stats_view;
2
3 CREATE TABLE car_stats (
4     car_id          INT PRIMARY KEY REFERENCES car(car_id) ON DELETE
5     CASCADE,
6     reg_number      VARCHAR(9) NOT NULL,
7     alert_count     BIGINT NOT NULL DEFAULT 0,
8     parking_count   BIGINT NOT NULL DEFAULT 0
9 );
10 INSERT INTO car_stats (car_id, reg_number, alert_count,
11     parking_count)
12 SELECT car_id, reg_number, alert_count, parking_count
13 FROM car_stats_view;
```

Listing 4: Создание и начальное заполнение таблицы статистики

3.2. Триггеры для таблицы автомобилей

При добавлении автомобиля в таблице статистики создаётся строка с нулевыми счётчиками. При изменении регистрационного номера обновляется его копия в **car_stats**. Перед удалением автомобиля удаляется соответствующая строка статистики.

```
1 CREATE OR REPLACE FUNCTION car_stats_change()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF TG_OP = 'INSERT' THEN
5         INSERT INTO car_stats (car_id, reg_number, alert_count,
6             parking_count)
7             VALUES (NEW.car_id, NEW.reg_number, 0, 0);
8         RETURN NEW;
9     ELSIF TG_OP = 'UPDATE' THEN
10        UPDATE car_stats
11        SET reg_number = NEW.reg_number
12        WHERE car_id = NEW.car_id;
```

```

12         RETURN NEW;
13     ELSIF TG_OP = 'DELETE' THEN
14         DELETE FROM car_stats
15         WHERE car_id = OLD.car_id;
16         RETURN OLD;
17     END IF;
18 END;
19 $$ LANGUAGE plpgsql;
20
21 CREATE TRIGGER car_stats_after_insert
22 AFTER INSERT ON car
23 FOR EACH ROW EXECUTE FUNCTION car_stats_change();
24
25 CREATE TRIGGER car_stats_after_reg_number_update
26 AFTER UPDATE OF reg_number ON car
27 FOR EACH ROW
28 WHEN (OLD.reg_number IS DISTINCT FROM NEW.reg_number)
29 EXECUTE FUNCTION car_stats_change();
30
31 CREATE TRIGGER car_stats_before_delete
32 BEFORE DELETE ON car
33 FOR EACH ROW EXECUTE FUNCTION car_stats_change();

```

Listing 5: Функция и триггеры для таблицы car

3.3. Триггер для парковочных сессий

Счётчик **parking_count** хранит число различных зон, поэтому при удалении одной сессии уменьшать его можно только тогда, когда у автомобиля больше не осталось сессий в этой же зоне. Функция на листинге ?? проверяет это условие до удаления строки.

```

1 CREATE OR REPLACE FUNCTION parking_session_stats_delete()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     IF NOT EXISTS (
5         SELECT 1
6         FROM parking_session
7         WHERE car_id = OLD.car_id
8             AND parking_zone_id = OLD.parking_zone_id
9             AND parking_session_id <> OLD.parking_session_id
10    ) THEN
11        UPDATE car_stats
12        SET parking_count = GREATEST(parking_count - 1, 0)
13        WHERE car_id = OLD.car_id;
14    END IF;
15    RETURN OLD;
16 END;
17 $$ LANGUAGE plpgsql;
18
19 CREATE TRIGGER parking_session_before_delete
20 BEFORE DELETE ON parking_session

```

```
21 FOR EACH ROW
22 EXECUTE FUNCTION parking_session_stats_delete();
```

Listing 6: Триггер удаления парковочной сессии

Отдельный триггер на удаление **parking_zone** не требуется. Внешний ключ **parking_session.parking_zone_id** настроен с правилом **ON DELETE CASCADE**; следовательно, удаление зоны удаляет связанные парковочные сессии, а триггер **parking_session_before_delete** корректирует статистику. Это предотвращает двойное уменьшение счётчика для одной зоны.

Место для снимков проверки триггеров: добавление автомобиля, изменение регистрационного номера и удаление последней парковочной сессии в зоне.

Рис. 6: Проверка работы триггеров